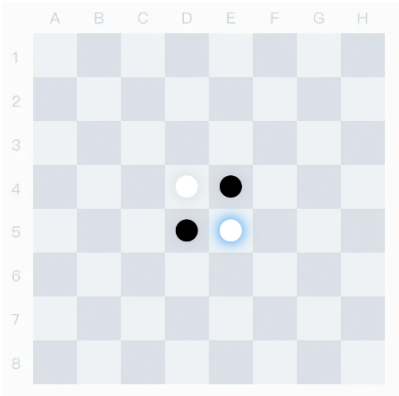
程序报告：黑白棋（Reversi）MCTS 智能体

安博源　学号：3230105806

1　问题重述



如上图所示，本实验旨在基于蒙特卡洛树搜索（MCTS）算法，这道题的主干可以概括为：给定一个黑白棋中途局面，让程序代替玩家完成下一步落子决策。程序面对的是标准 8×8 棋盘，需要遵守黑白棋的合法落子与翻转规则；如果当前存在可下位置，就要在这些合法步中选出质量最高的一步，如果无棋可下，则需要正确返回空步。

因此，真正要解决的核心不是把规则再实现一遍，而是在有限时间内做好”搜什么、怎么看、怎么选”。具体到实验要求，就是在 `AIPlayer` 的 `get_move(board)` 中，根据当前棋盘状态快速分析局面，输出一个合法坐标，并让智能体在与不同等级基准机器人的对战中尽量取得更好的胜负结果和终局优势。

2　设计思想

我的实现围绕 MCTS 的四大步骤（选择、扩展、模拟、回传）展开，前后共经历了三个版本：从标准 MCTS 出发，逐步用启发式知识替换其中”随机”和”盲目”的部分，最终得到一版以位棋盘为核心、搜索更加高效稳定的智能体。

2.1　基础实现：标准 MCTS 黑白棋智能体

第一版完全按照 MCTS 的教材流程实现。对于当前棋局，先建立根节点，然后在时间预算内反复执行”选择 → 扩展 → 模拟 → 回传”四步，最后选择根节点访问次数最多的动作作为落子。选择阶段采用经典的 UCT 公式，模拟阶段采用随机 rollout，回传阶段按 ± 1 回传胜负。

Listing 1: 基础版 MCTS 伪代码

```
函数 GET_MOVE(board):
```

```
    legal <- 当前合法动作
    若 legal 为空: 返回 None
    root <- 当前局面对应的根节点
    在时间限制内重复:
        node   <- TREE_POLICY(root)      # UCT 选择 + 扩展
        reward <- DEFAULT_POLICY(node)   # 随机 rollout
        BACKUP(node, reward)
    返回根节点访问次数最多的动作
```

实际测试发现：基础版可以战胜初级机器人，但净胜子数较少；面对中高级机器人时赢面不稳定，随机性很大。我分析原因有三点：

- 随机 rollout 噪声极大，在黑白棋这种”一步决定角的归属”的棋里常常误导统计；
- UCT 在给定时间内只能扩展出几千个节点，对中盘复杂局面覆盖度不够；
- 对黑白棋的结构知识（角、C/X 位、行动力、稳定性）没有任何利用。

2.2 第一轮优化：启发式评估、动作排序与残局精算

针对上面的问题，我保留 MCTS 的四步骨架，但把关键环节替换为启发式版本：

- **评估替代 rollout**：模拟阶段不再做完全随机的 rollout，而是用一个静态评估函数 `EVALUATE` 综合角 / 邻角 / 行动力 / 位置权重 / 前沿子 / 棋子差来估计局面；
- **动作排序**：对每一步按”是否送角””是否压缩对手行动力””位置权重”打分排序，让 MCTS 的扩展优先走更有潜力的分支；
- **残局精算**：对空位较少的局面直接切到 Alpha-Beta 搜索到终局，不再 rollout。

Listing 2: 第一轮优化伪代码

```
函数 EVALUATE(board, color):
    计算角点、危险位、行动力、前沿子、棋子差、位置权重等特征
    根据当前阶段加权求和
    返回评估值

函数 ORDER_ACTIONS(board, actions):
    对每个动作:
        执行动作并用 EVALUATE 估值
        若会送角给对手则降低分数
        若能压缩对手行动力则提高分数
    按分数排序后返回

函数 SEARCH_ENDGAME(board):
    当空位足够少时，用 Alpha-Beta 搜索至终局
    返回最优动作
```

这一版对初级和高级机器人胜率大幅提升，但仍有两个问题：**其一**，MCTS 维护 UCT 节点的成本很高，模拟里频繁复制棋盘列表非常慢，加上启发式评估和残局搜索同时参与，一局对弈常常

超过半小时；**其二**，MCTS 的统计量更新天然慢，面对某些风格鲜明的对手（实测是中级机器人）出现了”前中盘优势、后期连续翻车”的反常现象。

2.3 最终优化：位棋盘 + 迭代加深 PVS + 动态残局求解

继续追问”MCTS 框架在这里到底还剩下什么”，我发现：当 rollout 被评估替换、UCT 被”按评估选子节点”替换之后，整棵树其实已经退化成”对每个局面按分数挑最好分支”的递归搜索。这个退化的极限等价形式就是 **Negamax + Alpha-Beta**——可以把它视为”所有节点一次性扩展完、rollout 为静态评估、selection 为评分最大”的 MCTS。基于这个观察，最终版整体改为位棋盘 + 迭代加深 Alpha-Beta，保留 MCTS 之前调好的评估函数、动作排序与残局求解，并叠加若干工程优化：

1. **位棋盘表示**。用两个 64 位整数 `my`、`opp` 分别存黑 / 白棋子，合法着法 `_get_moves`（Dumb7Fill）、翻转 `_do_move`、子数统计 `_popcount` 全部用位运算完成，常数相比列表棋盘低一个数量级。
2. **迭代加深 + PVS（NegaScout）**。外层按深度从 1 逐步加深到 `max_depth`；内层对第一个着法按完整窗口搜索，其余着法先用零窗口 `[-α-1, -α]` 试探，仅当命中 (α, β) 时再以完整窗口重搜。相对朴素 Alpha-Beta 能再省下可观的节点。
3. **置换表（TT）**。键为 `(my, opp, passed)`，值为 `(depth, flag, value, best_move)`，其中 `flag`: 0 精确 / 1 下界 / 2 上界。TT 不仅用于截断子树，命中时保存的 `best_move` 会作为下一次迭代加深的首选着法（PV 排序）。
4. **残局精确求解**。末期把”到深度 0 返回静态评估”关掉，一直递归到双方均无合法步，返回真实胜负分 $\pm 10^6 +$ 净胜子（`_terminal_score`），残局完全精确。
5. **角点抢答**。决策开始时若任一合法步为角，直接取评分最高的角着法返回，省去搜索时间。

Listing 3: 最终版伪代码（对应 `main.py`）

```
函数 GET_MOVE(board):
    my, opp <- BOARD_TO_BITS(board)
    legal   <- GET_MOVES(my, opp)
    若 legal 为空:           返回 None
    若 legal 只有一个着法:   直接返回
    若 legal 包含角点:       按评分挑最好的角返回     # 角点抢答
    empties <- 空位数
    solve_threshold <- SOLVE_THRESHOLD(empties, legal_count)
    deadline <- now + TIME_BUDGET(empties)
    best    <- ORDER_MOVES(...)[0]
    对 depth = 1, 2, ..., DEPTH_LIMIT:
        move <- SEARCH_ROOT(my, opp, depth, best)    # PVS 根搜索
        若搜索完成: best <- move
        若已达残局深度: break
    (超时由 SearchTimeout 捕获，回退上一完整深度)
    返回 BIT_TO_COORD(best)
```

```
函数 SEARCH(my, opp, depth, alpha, beta, passed):
    查 TT: 命中且深度足够则按 flag 更新窗口或返回
    legal <- GET_MOVES(my, opp)
    若 legal 为空:
        若 passed: 返回 TERMINAL_SCORE
        否则交换双方, 递归 SEARCH(..., passed=True)
    empties <- 空位数;  solve_threshold <- SOLVE_THRESHOLD(empties, |legal|)
    若 depth <= 0 且 empties > solve_threshold:
        返回 EVALUATE(my, opp, legal)
    对 ORDER_MOVES(legal, tt_move) 中的每个 move:
        (new_my, new_opp) <- DO_MOVE(my, opp, move)
        若为首着: score <- 完整窗口  [-beta, -alpha]
        否则     : score <- 零窗口   [-alpha-1, -alpha]
                  若 alpha < score < beta: 再以 [-beta, -score] 重搜
        更新 best_score / alpha; 若 alpha >= beta 则剪枝
    写入 TT (flag: 精确 / 下界 / 上界)
    返回 best_score

函数 EVALUATE(my, opp):
    计算行动力、角、邻角压力、位置权重、前沿子、边、棋子差、奇偶性
    按阶段 (开局 / 中盘 / 残局) 加权求和
    返回评估值
```

走法排序（`_order_moves`）

Alpha-Beta 的剪枝效率强依赖排序。打分规则如下：

- 位置权重基础分：`POSITION_WEIGHTS[idx] * 8`；
- 角 `CORNERS`：`+200 000`；
- 置换表推荐 `tt_move`：`+100 000`；
- X 位 `-6 000`；C 位 `-2 500`；
- 落子后对手合法步数 $\times 80$（扣分）；
- 本步翻转数 $\times 8$（小幅正向，但故意压低，避免前中期贪吃）。

分阶段评估（`_evaluate`）

表 1: 评估函数分阶段权重

特征	开局 (> 40)	中盘 (17–40)	残局 (≤ 16)
行动力 mobility	10	8	5
角 corners	20	30	35
角邻压力 corner_pressure	12	10	4
位置权重 positional	4	3	2
前沿子 frontier	3	3	4
边 edges	1	2	4
棋子差 disc_diff	—	2	12
奇偶性 parity	—	—	8

动态时间与残局阈值

`_time_budget` 与 `_solve_threshold` 按剩余空位分段调度：

表 2: 动态时间预算与残局求解阈值

剩余空位 `empties`	时间预算 (s)	残局阈值	深度上限
> 50（开局）	1.2	—	`max_depth`= 12
41–50	2.2	—	12
29–40	3.5	—	12
17–28（中盘）	4.2	—	12
11–16（中后盘）	4.8	12	12
≤ 10（残局）	8.0	12	empties

追加规则：`empties` ≤ 14 且合法步数 ≤ 6 时残局阈值抬到 14，提前进入精确求解。

关键超参数

- `max_depth = 12`（非残局段深度硬上限）；
- `solve_threshold`：默认 12，贴近残局且分支收窄（empties ≤ 14 且 |legal| ≤ 6）时抬到 14；
- 时间预算：从开局的 1.2 s 单调增长到纯残局的 8.0 s，保证中盘留足深度、残局留足精确求解空间。

这些参数是在对弈测试中逐步调整得到的：固定其余部分、只动一组参数，与初级 / 中级 / 高级机器人分别进行先后手对局，综合比较胜负、净胜子数和单局耗时；最终保留在” 棋力、稳定性、总用时” 之间达到较好平衡的一组。

3 代码内容

核心实现位于 main.py，包含位棋盘常量、合法步 / 翻转位运算函数、评估与排序辅助函数，以及 AIPlayer 类的迭代加深 PVS 搜索。全文如下：

```python
import time


ALL_BITS = (1 << 64) - 1
NOT_A = 0xFEFEFEFEFEFEFEFE
NOT_H = 0x7F7F7F7F7F7F7F7F
INF = 10 ** 9
POPCOUNT_8 = tuple(bin(i).count("1") for i in range(256))

CORNERS = (1 << 0) | (1 << 7) | (1 << 56) | (1 << 63)
X_SQUARES = (1 << 9) | (1 << 14) | (1 << 49) | (1 << 54)
C_SQUARES = (
    (1 << 1) | (1 << 8) | (1 << 6) | (1 << 15) |
    (1 << 48) | (1 << 57) | (1 << 55) | (1 << 62)
)
EDGES = 0xFF818181818181FF
CORNER_NEIGHBORS = (
    (1 << 0, (1 << 1) | (1 << 8) | (1 << 9)),
    (1 << 7, (1 << 6) | (1 << 14) | (1 << 15)),
    (1 << 56, (1 << 48) | (1 << 49) | (1 << 57)),
    (1 << 63, (1 << 54) | (1 << 55) | (1 << 62)),
)

POSITION_WEIGHTS = (
    120, -20, 20, 5, 5, 20, -20, 120,
    -20, -50, -2, -2, -2, -2, -50, -20,
    20, -2, 15, 3, 3, 15, -2, 20,
    5, -2, 3, 3, 3, 3, -2, 5,
    5, -2, 3, 3, 3, 3, -2, 5,
    20, -2, 15, 3, 3, 15, -2, 20,
    -20, -50, -2, -2, -2, -2, -50, -20,
    120, -20, 20, 5, 5, 20, -20, 120,
)


class SearchTimeout(Exception):
    pass


def _iter_bits(bb):
    while bb:
        move = bb & -bb
        yield move
        bb ^= move

```

```python

def _get_moves(my, opp):
    empty = ALL_BITS ^ (my | opp)
    moves = 0

    m = NOT_A
    c = ((my << 1) & m) & opp
    c |= ((c << 1) & m) & opp
    c |= ((c << 1) & m) & opp
    c |= ((c << 1) & m) & opp
    c |= ((c << 1) & m) & opp
    c |= ((c << 1) & m) & opp
    moves |= ((c << 1) & m) & empty

    m = NOT_H
    c = ((my >> 1) & m) & opp
    c |= ((c >> 1) & m) & opp
    c |= ((c >> 1) & m) & opp
    c |= ((c >> 1) & m) & opp
    c |= ((c >> 1) & m) & opp
    c |= ((c >> 1) & m) & opp
    moves |= ((c >> 1) & m) & empty

    c = (my << 8) & opp
    c |= (c << 8) & opp
    c |= (c << 8) & opp
    c |= (c << 8) & opp
    c |= (c << 8) & opp
    c |= (c << 8) & opp
    moves |= (c << 8) & empty

    c = (my >> 8) & opp
    c |= (c >> 8) & opp
    c |= (c >> 8) & opp
    c |= (c >> 8) & opp
    c |= (c >> 8) & opp
    c |= (c >> 8) & opp
    moves |= (c >> 8) & empty

    m = NOT_A
    c = ((my << 9) & m) & opp
    c |= ((c << 9) & m) & opp
    c |= ((c << 9) & m) & opp
    c |= ((c << 9) & m) & opp
    c |= ((c << 9) & m) & opp
    c |= ((c << 9) & m) & opp
    moves |= ((c << 9) & m) & empty

    m = NOT_H
    c = ((my << 7) & m) & opp
```

```
    c |= ((c << 7) & m) & opp
    c |= ((c << 7) & m) & opp
    c |= ((c << 7) & m) & opp
    c |= ((c << 7) & m) & opp
    c |= ((c << 7) & m) & opp
    moves |= ((c << 7) & m) & empty

    m = NOT_A
    c = ((my >> 7) & m) & opp
    c |= ((c >> 7) & m) & opp
    c |= ((c >> 7) & m) & opp
    c |= ((c >> 7) & m) & opp
    c |= ((c >> 7) & m) & opp
    c |= ((c >> 7) & m) & opp
    moves |= ((c >> 7) & m) & empty

    m = NOT_H
    c = ((my >> 9) & m) & opp
    c |= ((c >> 9) & m) & opp
    c |= ((c >> 9) & m) & opp
    c |= ((c >> 9) & m) & opp
    c |= ((c >> 9) & m) & opp
    c |= ((c >> 9) & m) & opp
    moves |= ((c >> 9) & m) & empty

    return moves & ALL_BITS


def _do_move(my, opp, move):
    flipped = 0

    f = 0
    p = (move << 1) & NOT_A
    while p & opp:
        f |= p
        p = (p << 1) & NOT_A
    if p & my:
        flipped |= f

    f = 0
    p = (move >> 1) & NOT_H
    while p & opp:
        f |= p
        p = (p >> 1) & NOT_H
    if p & my:
        flipped |= f

    f = 0
    p = (move << 8) & ALL_BITS
    while p & opp:
```

```
        f |= p
        p = (p << 8) & ALL_BITS
    if p & my:
        flipped |= f

    f = 0
    p = move >> 8
    while p & opp:
        f |= p
        p >>= 8
    if p & my:
        flipped |= f

    f = 0
    p = (move << 9) & NOT_A & ALL_BITS
    while p & opp:
        f |= p
        p = (p << 9) & NOT_A & ALL_BITS
    if p & my:
        flipped |= f

    f = 0
    p = (move << 7) & NOT_H & ALL_BITS
    while p & opp:
        f |= p
        p = (p << 7) & NOT_H & ALL_BITS
    if p & my:
        flipped |= f

    f = 0
    p = (move >> 7) & NOT_A
    while p & opp:
        f |= p
        p = (p >> 7) & NOT_A
    if p & my:
        flipped |= f

    f = 0
    p = (move >> 9) & NOT_H
    while p & opp:
        f |= p
        p = (p >> 9) & NOT_H
    if p & my:
        flipped |= f

    return (my | move | flipped) & ALL_BITS, opp & ~flipped


def _board_to_bits(board):
    black = 0
```

```python
    white = 0
    for row in range(8):
        base = row << 3
        for col, piece in enumerate(board._board[row]):
            bit = 1 << (base + col)
            if piece == "X":
                black |= bit
            elif piece == "O":
                white |= bit
    return black, white


def _bit_to_coord(bit):
    idx = bit.bit_length() - 1
    row = idx >> 3
    col = idx & 7
    return chr(ord("A") + col) + str(row + 1)


def _adjacent(bb):
    return (
        ((bb << 1) & NOT_A) |
        ((bb >> 1) & NOT_H) |
        ((bb << 8) & ALL_BITS) |
        (bb >> 8) |
        ((bb << 9) & NOT_A & ALL_BITS) |
        ((bb << 7) & NOT_H & ALL_BITS) |
        ((bb >> 7) & NOT_A) |
        ((bb >> 9) & NOT_H)
    ) & ALL_BITS


def _scaled_diff(a, b):
    total = a + b
    if total == 0:
        return 0
    return 100 * (a - b) // total


def _popcount(x):
    return (
        POPCOUNT_8[x & 0xFF] +
        POPCOUNT_8[(x >> 8) & 0xFF] +
        POPCOUNT_8[(x >> 16) & 0xFF] +
        POPCOUNT_8[(x >> 24) & 0xFF] +
        POPCOUNT_8[(x >> 32) & 0xFF] +
        POPCOUNT_8[(x >> 40) & 0xFF] +
        POPCOUNT_8[(x >> 48) & 0xFF] +
        POPCOUNT_8[(x >> 56) & 0xFF]
    )
```

```python


def _positional_score(my, opp):
    score = 0
    for bb, sign in ((my, 1), (opp, -1)):
        current = bb
        while current:
            bit = current & -current
            score += sign * POSITION_WEIGHTS[bit.bit_length() - 1]
            current ^= bit
    return score


class AIPlayer:
    def __init__(self, color):
        self.color = color
        self.max_depth = 12
        self.tt = {}
        self.deadline = 0.0
        self.nodes = 0
        self.best_move = 0

    def _time_budget(self, empties):
        if empties > 50:
            return 1.2
        if empties > 40:
            return 2.2
        if empties > 28:
            return 3.5
        if empties > 16:
            return 4.2
        if empties > 10:
            return 4.8
        return 8.0

    def _solve_threshold(self, empties, legal_count):
        if empties <= 14 and legal_count <= 6:
            return 14
        return 12

    def _depth_limit(self, empties, solve_threshold):
        if empties <= solve_threshold:
            return empties
        return self.max_depth

    def get_move(self, board):
        player_name = "黑棋" if self.color == "X" else "白棋"
        print("请等一会，对方 {}-{} 正在思考中...".format(player_name, self.color))

        black, white = _board_to_bits(board)
```

```
        my = black if self.color == "X" else white
        opp = white if self.color == "X" else black
        legal_moves = _get_moves(my, opp)
        if legal_moves == 0:
            return None
        if legal_moves & (legal_moves - 1) == 0:
            return _bit_to_coord(legal_moves)

        corner_moves = legal_moves & CORNERS
        if corner_moves:
            corner_list = self._order_moves(my, opp, corner_moves, 0)
            self.best_move = corner_list[0]
            return _bit_to_coord(corner_list[0])

        empties = 64 - _popcount(my | opp)
        legal_count = _popcount(legal_moves)
        solve_threshold = self._solve_threshold(empties, legal_count)
        budget = min(self._time_budget(empties), 55.0)
        self.deadline = time.perf_counter() + budget
        self.nodes = 0
        self.tt.clear()

        ordered_moves = self._order_moves(my, opp, legal_moves, 0)
        best_move = ordered_moves[0]
        completed_depth = 0

        try:
            max_depth = min(self._depth_limit(empties, solve_threshold), empties)
            for depth in range(1, max_depth + 1):
                move, _ = self._search_root(my, opp, depth, best_move)
                if move:
                    best_move = move
                    completed_depth = depth
                if empties <= solve_threshold and completed_depth >= empties:
                    break
        except SearchTimeout:
            pass

        self.best_move = best_move
        return _bit_to_coord(best_move)

    def _check_time(self):
        self.nodes += 1
        if self.nodes & 1023 == 0 and time.perf_counter() >= self.deadline:
            raise SearchTimeout

    def _terminal_score(self, my, opp):
        diff = _popcount(my) - _popcount(opp)
        if diff > 0:
            return 1000000 + diff
```

```
        if diff < 0:
            return -1000000 + diff
        return 0

    def _corner_pressure(self, my, opp, occupied):
        score = 0
        for corner, neighbors in CORNER_NEIGHBORS:
            if occupied & corner:
                continue
            score += _popcount(opp & neighbors) - _popcount(my & neighbors)
        return score

    def _frontier_score(self, my, opp, occupied):
        empty = ALL_BITS ^ occupied
        frontier = _adjacent(empty)
        my_frontier = _popcount(my & frontier)
        opp_frontier = _popcount(opp & frontier)
        return _scaled_diff(opp_frontier, my_frontier)

    def _evaluate(self, my, opp, my_moves):
        occupied = my | opp
        empties = 64 - _popcount(occupied)
        opp_moves = _get_moves(opp, my)

        mobility = _scaled_diff(_popcount(my_moves), _popcount(opp_moves))
        disc_diff = _scaled_diff(_popcount(my), _popcount(opp))
        positional = _positional_score(my, opp)
        corners = 25 * (_popcount(my & CORNERS) - _popcount(opp & CORNERS))
        edges = _popcount(my & EDGES) - _popcount(opp & EDGES)
        frontier = self._frontier_score(my, opp, occupied)
        corner_pressure = self._corner_pressure(my, opp, occupied)
        parity = 1 if empties & 1 else -1

        if empties > 40:
            return (
                10 * mobility +
                20 * corners +
                12 * corner_pressure +
                4 * positional +
                3 * frontier +
                edges
            )

        if empties > 16:
            return (
                8 * mobility +
                30 * corners +
                10 * corner_pressure +
                3 * frontier +
                3 * positional +
```

```
                2 * edges +
                2 * disc_diff
            )

        return (
            5 * mobility +
            35 * corners +
            4 * corner_pressure +
            4 * frontier +
            2 * positional +
            4 * edges +
            12 * disc_diff +
            8 * parity
        )

    def _order_moves(self, my, opp, moves, tt_move):
        ordered = []
        for move in _iter_bits(moves):
            idx = move.bit_length() - 1
            score = POSITION_WEIGHTS[idx] * 8
            if move & CORNERS:
                score += 200000
            if move == tt_move:
                score += 100000
            if move & X_SQUARES:
                score -= 6000
            elif move & C_SQUARES:
                score -= 2500

            new_my, new_opp = _do_move(my, opp, move)
            score -= _popcount(_get_moves(new_opp, new_my)) * 80
            score += (_popcount(new_my) - _popcount(my)) * 8
            ordered.append((score, move))

        ordered.sort(reverse=True)
        return [move for _, move in ordered]

    def _search_root(self, my, opp, depth, preferred_move):
        legal_moves = _get_moves(my, opp)
        ordered_moves = self._order_moves(my, opp, legal_moves, preferred_move)
        alpha = -INF
        beta = INF
        best_score = -INF
        best_move = ordered_moves[0]

        for i, move in enumerate(ordered_moves):
            self._check_time()
            new_my, new_opp = _do_move(my, opp, move)
            if i == 0:
                score = -self._search(new_opp, new_my, depth - 1, -beta, -alpha, False)
```

```
            else:
                score = -self._search(new_opp, new_my, depth - 1, -alpha - 1, -alpha, False)
                if alpha < score < beta:
                    score = -self._search(new_opp, new_my, depth - 1, -beta, -score, False)
            if score > best_score:
                best_score = score
                best_move = move
            if score > alpha:
                alpha = score

        return best_move, best_score

    def _search(self, my, opp, depth, alpha, beta, passed):
        self._check_time()
        alpha_orig = alpha
        key = (my, opp, passed)
        entry = self.tt.get(key)
        tt_move = 0

        if entry is not None:
            stored_depth, flag, value, tt_move = entry
            if stored_depth >= depth:
                if flag == 0:
                    return value
                if flag == 1 and value > alpha:
                    alpha = value
                elif flag == 2 and value < beta:
                    beta = value
                if alpha >= beta:
                    return value

        legal_moves = _get_moves(my, opp)
        if legal_moves == 0:
            if passed:
                return self._terminal_score(my, opp)
            value = -self._search(opp, my, depth, -beta, -alpha, True)
            self.tt[key] = (depth, 0, value, 0)
            return value

        occupied = my | opp
        empties = 64 - _popcount(occupied)
        solve_threshold = self._solve_threshold(empties, _popcount(legal_moves))
        if depth <= 0 and empties > solve_threshold:
            return self._evaluate(my, opp, legal_moves)

        best_score = -INF
        best_move = 0
        ordered_moves = self._order_moves(my, opp, legal_moves, tt_move)

        for i, move in enumerate(ordered_moves):
```

```python
            new_my, new_opp = _do_move(my, opp, move)
            if i == 0:
                score = -self._search(new_opp, new_my, depth - 1, -beta, -alpha, False)
            else:
                score = -self._search(new_opp, new_my, depth - 1, -alpha - 1, -alpha, False)
                if alpha < score < beta:
                    score = -self._search(new_opp, new_my, depth - 1, -beta, -score, False)
            if score > best_score:
                best_score = score
                best_move = move
            if score > alpha:
                alpha = score
            if alpha >= beta:
                break

        flag = 0
        if best_score <= alpha_orig:
            flag = 2
        elif best_score >= beta:
            flag = 1
        self.tt[key] = (depth, flag, best_score, best_move)
        return best_score
```

Listing 4: main.py 完整代码

4 实验结果

分别与初级、中级、高级机器人进行先手 / 后手对弈，结果如下。

图 1: 初级机器人：先手 / 后手对弈结果

图 2: 中级机器人：先手 / 后手对弈结果

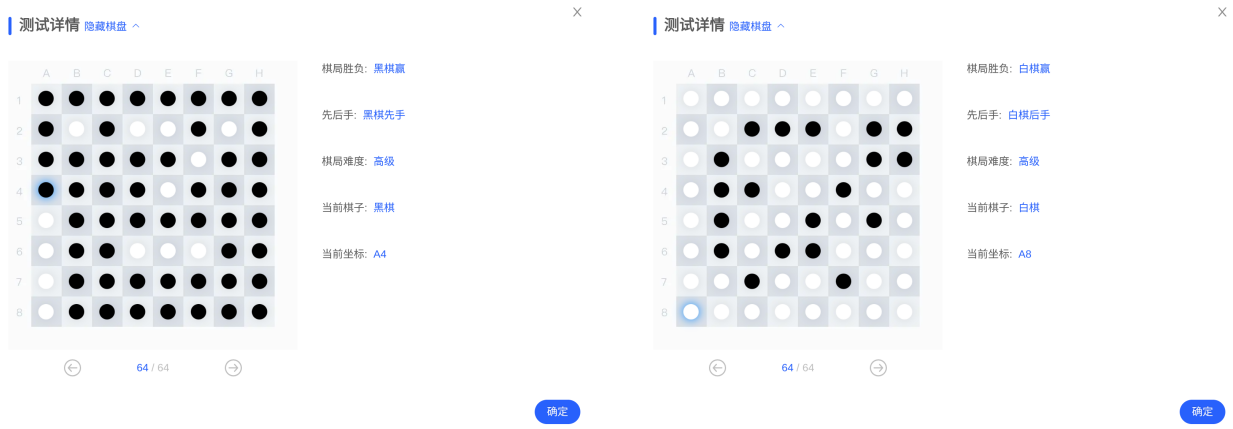


图 3: 高级机器人：先手 / 后手对弈结果

从实验结果可见，最终版本在所有难度 × 先后手的组合下均稳定取胜，多数局次保持较大的净胜子优势。

5 总结

这次实验让我从”标准 MCTS”出发，一路把随机 rollout 换成静态评估、把 UCT 选择退化为 α-β 剪枝，最终得到一个工程化程度更高、稳定性也更好的搜索智能体。一开始我以为只要把 MCTS 的四步写对、再加上一点启发式就够了；直到真正去分析”每 1 秒内到底跑了多少次有效模拟”，才理解 MCTS 在黑白棋这种评估函数相对成熟的问题上并不是最省算力的选择——真正有效的 MCTS 变体最后几乎都变成了 Negamax + 评估，区别只在”搜索树的展开策略”。这次手把手走完这条退化路径，对我理解博弈搜索的帮助很大。

实现层面上，下面这几件事对最终棋力影响最大，且很多都是代码上看着小、实际提升很大的改动：

- **位棋盘压低了基本常数**。合法步和翻转全部用位运算之后，同样深度的搜索速度相比列表棋盘约快一个数量级。没有这一步，后面所有优化都撑不开搜索深度。

- **PVS（NegaScout）对黑白棋特别有效**。因为黑白棋里大部分非主变着法的评分会迅速掉下来，零窗口探测命中率很高，重搜代价相对很小；我粗测同样 4 s 预算下，PVS 比朴素 α-β 多搜 1–2 层。
- **置换表既剪枝又排序**。命中 TT 时保存的 `best_move` 作为下一层迭代加深的首选着法，PV 基本能一次性跑通，剪枝效率远比单纯”精确/下界/上界”命中更高。
- **动态时间预算比固定每步 1 s 合理得多**。开局走平凡位置只花 1.2 s，省下的预算给残局，让残局精确求解阈值可以放到 12–14，决定性收益来自末期。
- **角点抢答是”规则级”优化**。出现可下角时几乎不存在需要放弃它的局面，直接返回省下的那几秒又可以变成后续几步的深度。
- **送角惩罚要”极端”**。X 位 $-6\,000$、C 位 $-2\,500$ 看起来很夸张，但正是这种量级才能把”送角”稳稳地压到排序末尾、让 α-β 早早剪枝。

仍有可以继续做的方向：(1) 用 **Zobrist 哈希**替代当前（`my`，`opp`，`passed`）三元组作为 TT 键，把键长从 ~ 150 位压到 64 位，`dict` 命中检查更快；(2) 评估函数里的”稳定性”目前只是用边格计数近似，可以引入真正的稳定棋子传播算法，进一步压缩启发式噪声；(3) 前若干步换成 **开局库**，把搜索时间让给更关键的中后盘；(4) 走法排序和评估权重还有调参空间，但手调接近边际了，更合理的方式是构造自对弈数据集，让权重由数据拟合。

总的来说，这次实验让我完整经历了”按教材照抄 MCTS → 发现瓶颈 → 把 MCTS 的每一步替换掉 → 在工程细节里打磨”的完整过程。真正的收获是：博弈搜索的胜负常常不在”想法新不新”，而在”想法能不能被快到极致地执行”——位棋盘、置换表、PVS、走法排序、动态时间分配这些看起来偏工程的东西，恰恰是把算法从”能跑”推到”能赢”的决定性因素。